

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО.
УТИЛИТА ДЛЯ ИНВЕРТИРОВАНИЯ СЛОВ»
Вариант 8

Студент:

А.П. Бельская

Преподаватель:

Д.В. Куприянова

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель лабораторной работы – освоить практическую работу в выбранной IDE при разработке небольшого объектно-ориентированного Java-приложения, проверить настройку проекта, запуск и отладку программы, а также показать применение MVC на задаче варианта 8.

По заданию требуется создать приложение с графическим интерфейсом: пользователь вводит предложение в отдельном окне, после чего программа выводит на главной форме исходную строку и строку, в которой каждое слово записано наоборот. При повторном вводе должны восстанавливаться последние данные, а некорректный ввод должен сопровождаться сообщением об ошибке. Выполнение работы разбито на следующие этапы:

1. Проверить установленную среду IntelliJ IDEA и настроить Java-проект для выполнения варианта.
2. Продемонстрировать настройку JDK, структуру Maven-проекта и возможность запуска простого Java-кода.
3. Установить расширения, применимые для UML-диаграмм и структурного описания системы.
4. Реализовать главное окно, модальное окно ввода и обработку пользовательского действия «Ввести данные».
5. Разработать алгоритм инвертирования слов с сохранением разделителей и знаков препинания.
6. Организовать код по MVC и реализовать активную модель через список наблюдателей.
7. Проверить запуск, работу отладчика, восстановление последнего ввода и реакцию приложения на пустую строку.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Интегрированная среда разработки и её компоненты

IDE используется не только как текстовый редактор, а как единая рабочая среда. В ней находятся дерево проекта, редактор исходных файлов, средства сборки, запуск и отладка, поиск по коду, работа с зависимостями и расширениями. Для Java это удобно тем, что ошибки импорта, синтаксиса и типов видны ещё до запуска программы.

2.2 Редактирование и навигация в IntelliJ IDEA

При написании кода использовались стандартные возможности редактора IntelliJ IDEA: подсветка синтаксиса, автодополнение имён классов и методов, быстрый импорт Swing-компонентов, переход между классами проекта и форматирование исходного текста. Это ускоряет разработку и снижает количество механических ошибок.

2.3 Сборка, запуск и отладка Java-приложения

После сборки исходные файлы .java превращаются в .class-файлы, которые выполняет JVM. В данной работе проект открыт как Maven-проект, поэтому IntelliJ IDEA видит стандартную структуру src/main/java. Для проверки работы IDE использовались запуск через Run и отладка через Debug с остановкой на выбранной строке.

2.4 Паттерн MVC

Паттерн MVC нужен для разделения обязанностей между классами. Модель не должна знать, как именно нарисовано окно; представление не должно выполнять расчёты; контроллер связывает событие от пользователя с изменением состояния модели. Для небольшой лабораторной работы такое разделение особенно удобно, потому что по коду сразу видно, где находится логика, а где интерфейс.

2.5 Активная и пассивная модель

В пассивном варианте MVC контроллер после каждого изменения явно обновляет окно. В активном варианте модель сама сообщает заинтересованным объектам, что её данные изменились. В приложении для варианта 8 используется второй подход: WordInversionModel хранит слушателей ModelListener и вызывает их после пересчёта результата.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разрабатываемая программа решает текстовую задачу: из введённого предложения формируется новая строка, где символы каждого слова идут в обратном порядке. Например, строка «Привет, мир!» преобразуется в «тевирП, рим!». Пользователь взаимодействует с программой через главное окно и отдельное окно ввода.

3.2 Установка и настройка IntelliJ IDEA

Работа выполнялась в IntelliJ IDEA. Перед началом разработки была проверена установка IDE на компьютере, затем в параметрах проекта был выбран JDK 17. Это важно, потому что без настроенного SDK среда не сможет корректно компилировать и запускать Java-классы.

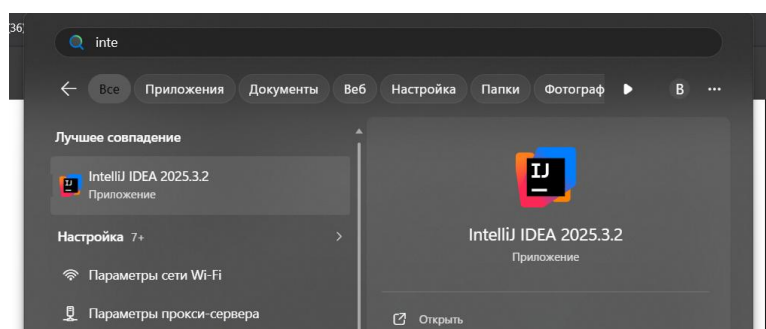


Рисунок 3.1 – Наличие IntelliJ IDEA на компьютере

На рисунке видно, что выбранная среда разработки уже установлена и доступна для запуска на компьютере.

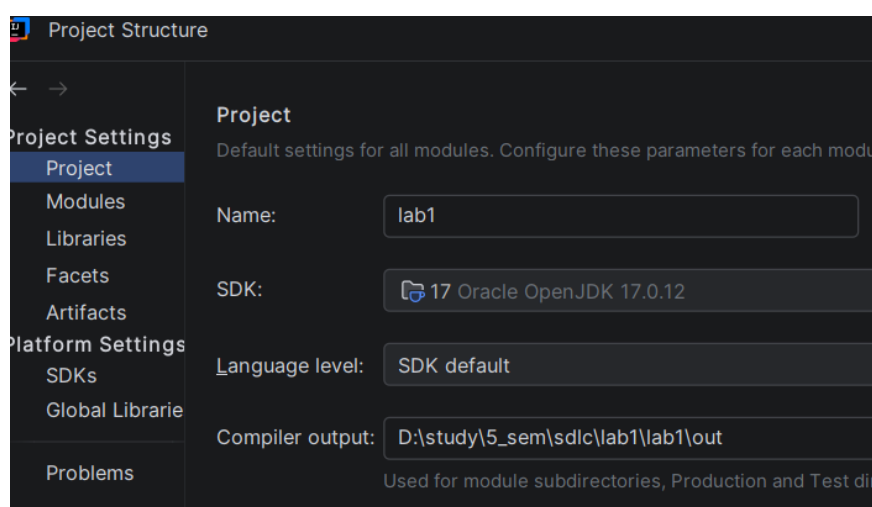


Рисунок 3.2 – Настройка Project SDK и каталога вывода

Параметр SDK показывает, какой JDK будет использоваться при компиляции. Это один из обязательных шагов перед запуском Java-проекта.

3.3 Создание проекта и настройка расширений для UML и SysML v2

Для лабораторной работы создан отдельный проект lab1. Maven-структура позволяет отделить исходный код от служебных файлов IDE и результатов сборки. Для визуализации классов был подключён PlantUML-плагин. Прямой полноценный SysML v2-инструмент в рамках проекта не использовался, поэтому состав приложения дополнительно описан текстовым структурным фрагментом.

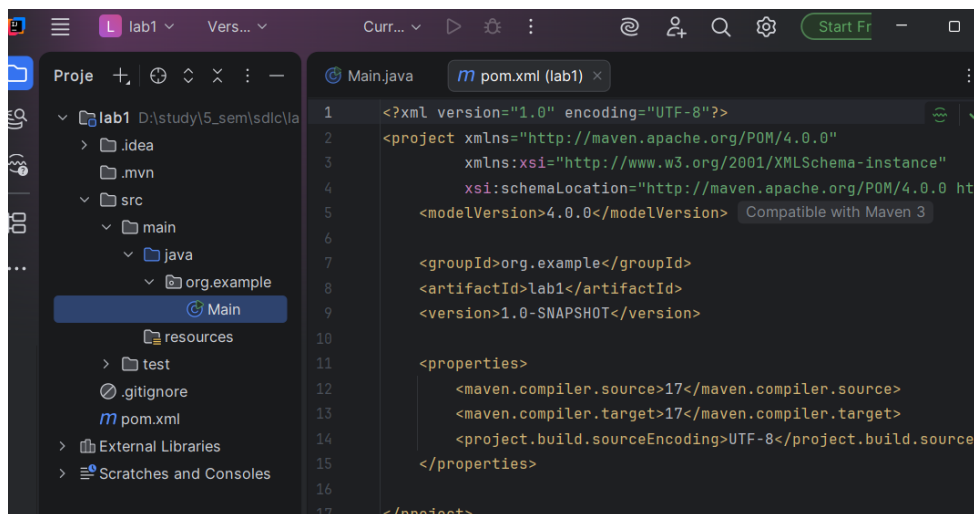


Рисунок 3.3 – Структура Maven-проекта в IntelliJ IDEA

В дереве проекта выделены pom.xml, каталог src/main/java и пакет org.example, где размещены классы приложения.

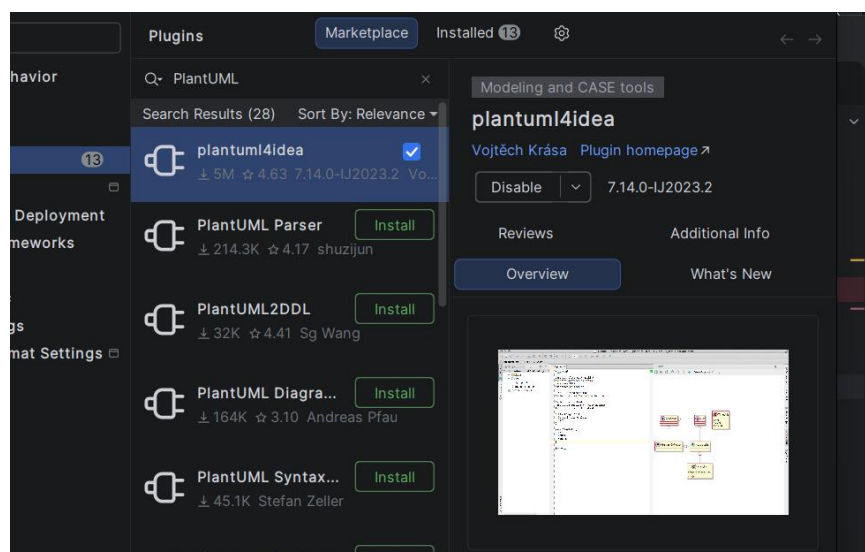


Рисунок 3.4 – Установленный плагин plantuml4idea для UML

Установленный плагин используется для просмотра PlantUML-диаграмм прямо в среде разработки.

Ниже приведено краткое структурное описание, показывающее состав приложения и направления взаимодействия между частями:

```
package WordInversionApplication {
  part def WordInversionModel;
  part def MainFrame;
  part def InputDialog;
  part def WordInversionController;

  part application {
    part model : WordInversionModel;
    part view : MainFrame;
    part dialog : InputDialog;
    part controller : WordInversionController;
    connect view to controller;
    connect controller to model;
    connect model to view;
  }
}
```

3.4 Демонстрация функциональных возможностей IDE и отладчика

Перед переносом основного кода был выполнен небольшой проверочный пример. Он нужен не для решения варианта, а для демонстрации возможностей IDE: запуск программы, вывод в консоль, установка breakpoint и просмотр значений переменных во время пошагового выполнения.

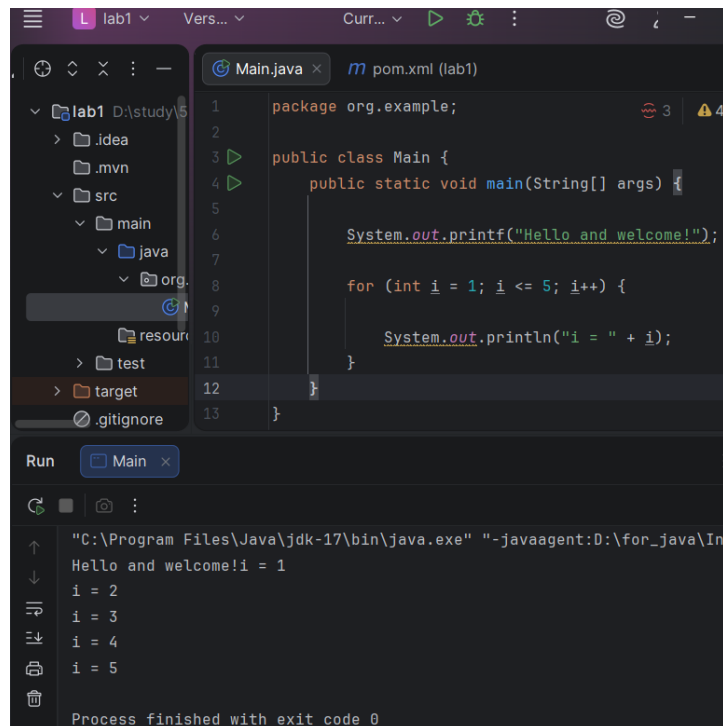


Рисунок 3.5 – Запуск простого Java-примера в IntelliJ IDEA

Консольный вывод подтверждает, что конфигурация запуска работает и программа завершается без ошибки.

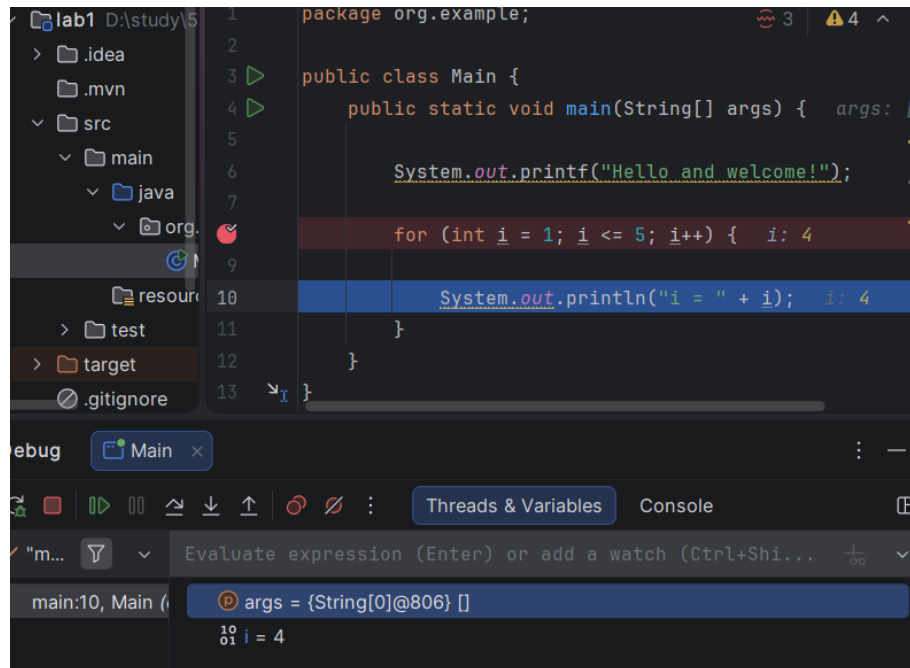


Рисунок 3.6 – Остановка программы в отладчике IntelliJ IDEA

Панель Debug показывает текущую строку выполнения и значения переменных, что позволяет проверять поведение программы пошагово.

3.5 Структура проекта и каркас MVC с активной моделью

Основная программа разбита на несколько классов. Main создаёт модель, главное окно и контроллер. WordInversionController принимает событие от кнопки и решает, нужно ли передавать строку в модель. WordInversionModel хранит состояние и выполняет преобразование текста. MainFrame и InputDialog отвечают за отображение данных и ввод пользователя.

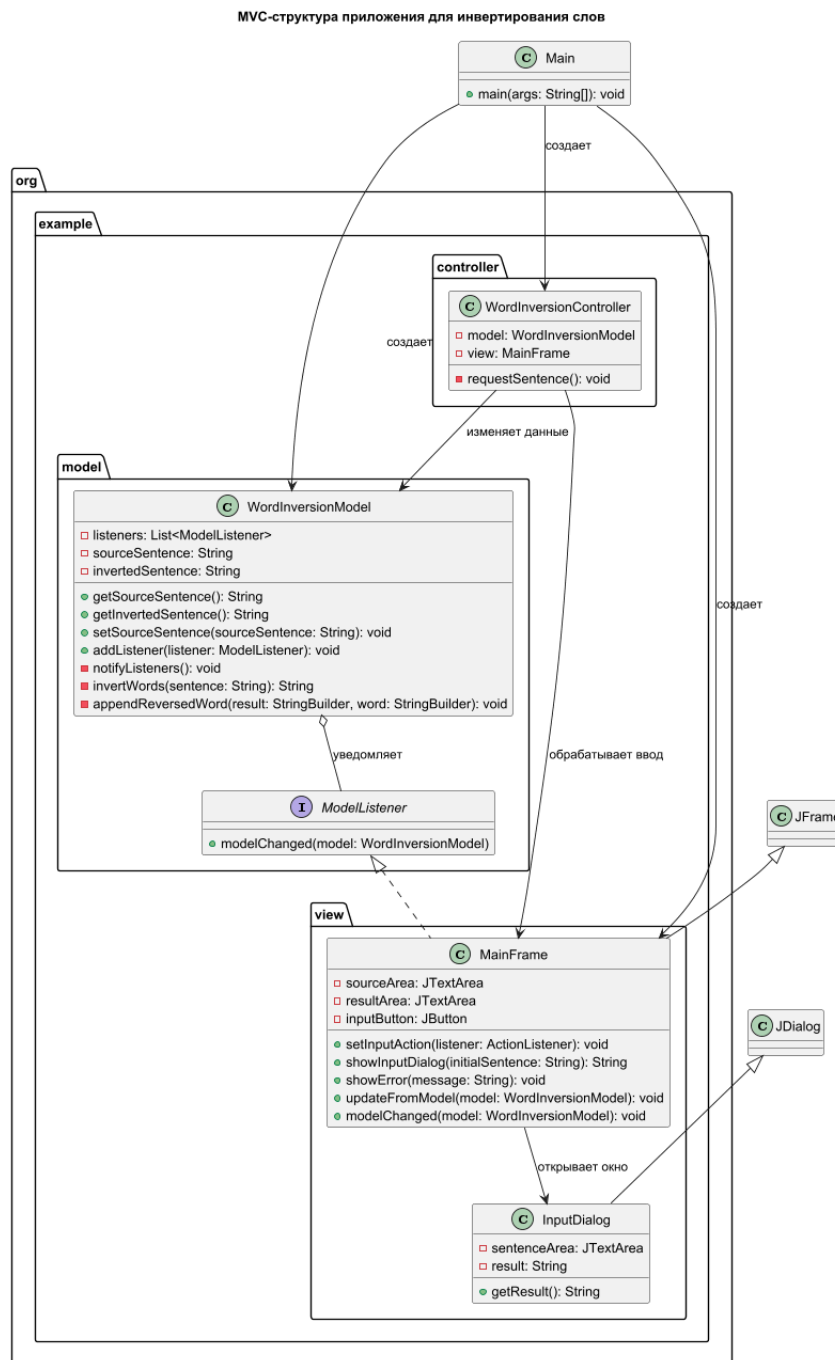


Рисунок 3.7 – UML-диаграмма классов приложения

Диаграмма показывает, что Main создаёт основные объекты, контроллер связывает форму с моделью, а модель уведомляет представление через интерфейс ModelListener.

3.6 Модель

В модели находится предметная логика, поэтому именно WordInversionModel отвечает за преобразование строки. Сначала сохраняется исходное предложение, затем вычисляется результат. При проходе по строке последовательности букв и цифр считаются словами, а остальные символы

выступают разделителями. После изменения состояния модель уведомляет слушателей.

3.7 Представление

Графический интерфейс реализован средствами Swing. Главное окно содержит две текстовые области: в первой показывается последний ввод, во второй - результат обработки. Окно InputDialog открывается поверх главного окна и блокирует дальнейшую работу с ним до нажатия ОК или «Отмена».

3.8 Контроллер

Контроллер является связующим слоем. Он не инвертирует строку самостоятельно, а только получает данные из окна ввода, проверяет их и передаёт модели. Такой подход сохраняет разделение обязанностей: проверка пользовательского действия находится в контроллере, а обработка текста - в модели.

3.9 Запоминание данных и обработка ошибок

Требование о запоминании данных выполнено за счёт хранения исходной строки в модели. Когда пользователь снова открывает окно ввода, оно получает уже сохранённый текст. Проверка ошибки сделана на уровне контроллера: если после удаления пробелов строка пустая, расчёт не выполняется и показывается сообщение.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В результате получено Swing-приложение для варианта 8. Оно открывает отдельное окно ввода, отображает исходный текст и преобразованную строку, восстанавливает последний ввод и корректно реагирует на пустое предложение. При изменении данных модель уведомляет главное окно, поэтому интерфейс обновляется без прямого ручного вызова из контроллера.

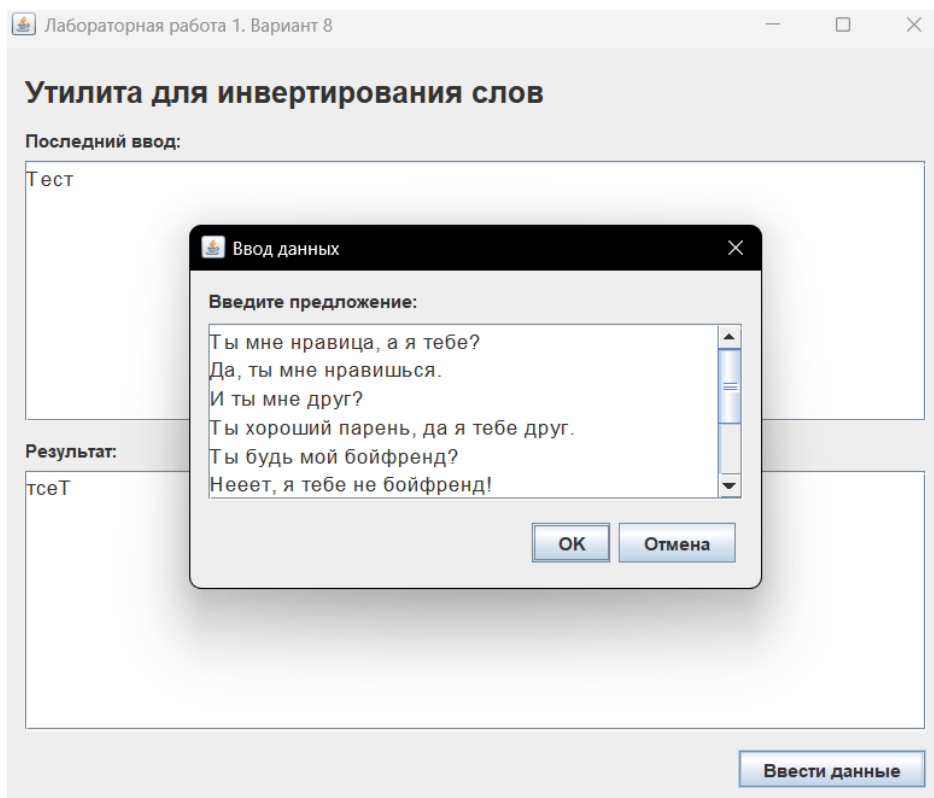


Рисунок 4.1 – Окно ввода данных с восстановленным последним предложением

Окно ввода содержит ранее введенный текст, что демонстрирует выполнение требования о восстановлении последних данных.

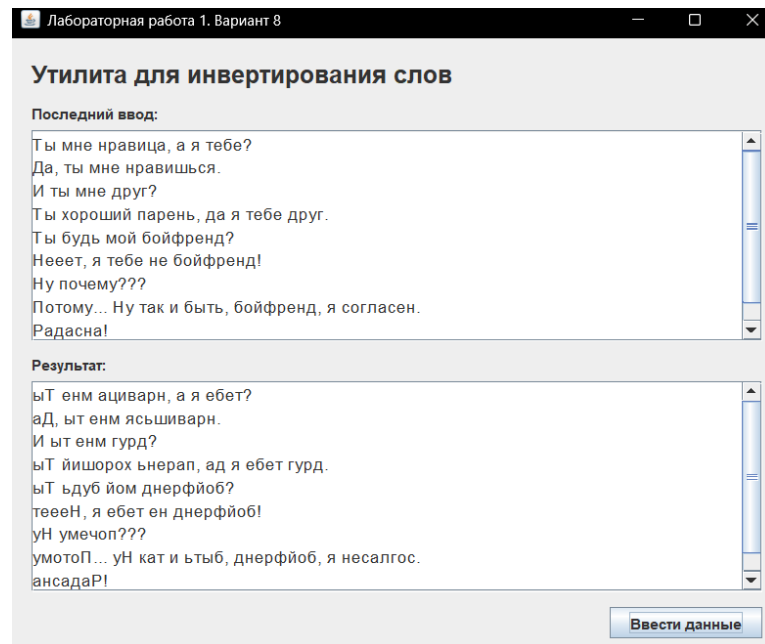


Рисунок 4.2 – Главная форма с результатом инвертирования слов

В верхней области находится исходное предложение, а в нижней – результат, где каждое слово записано в обратном порядке.

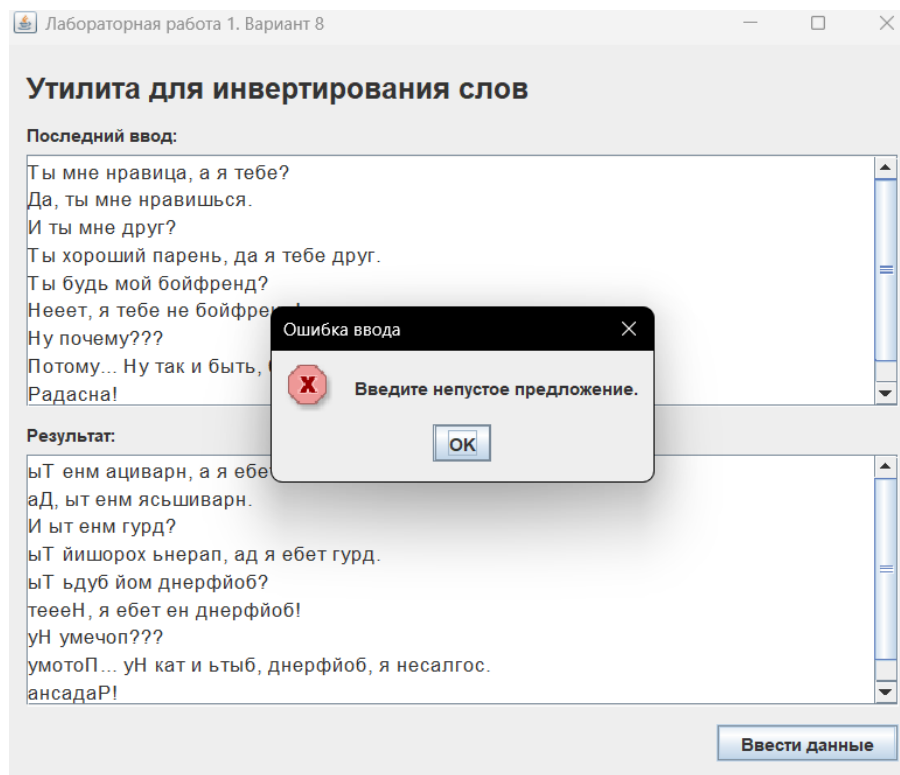


Рисунок 4.3 – Сообщение об ошибке при пустом вводе

Сообщение появляется при попытке подтвердить пустую строку, поэтому некорректные данные не попадают в модель.

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

IDE обычно включает редактор исходного кода, подсветку синтаксиса, автодополнение, инструменты навигации, компилятор или интеграцию со сборщиком, систему запуска, отладчик, терминал, работу с зависимостями, поддержку контроля версий и менеджер плагинов.

Вопрос 2 Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся создание классов и пакетов, редактирование методов, форматирование кода, быстрый импорт нужных классов, переход к объявлению, поиск по проекту, просмотр использований, переименование элементов и анализ предупреждений IDE.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

При сборке исходные Java-файлы преобразуются в байт-код. В IntelliJ IDEA для этого можно использовать Maven или встроенную команду Build Project. Запуск выполняется через конфигурацию, связанную с классом Main, где расположен метод main.

Вопрос 4 Что такое MVC?

MVC – это способ организации программы, при котором данные и логика отделены от интерфейса и обработки пользовательских действий. В данной работе WordInversionModel выполняет роль модели, MainFrame и InputDialog относятся к представлению, а WordInversionController выполняет роль контроллера.

Вопрос 5 Чем отличается активная модель от пассивной?

Пассивная модель только хранит данные, а контроллер сам решает, когда обновлять интерфейс. Активная модель после изменения состояния генерирует уведомление для подписчиков. В программе это реализовано через ModelListener: MainFrame подписан на модель и обновляется после вызова notifyListeners.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была проверена установка IntelliJ IDEA, настроен Java-проект с JDK 17, изучены запуск и отладка программы, а также показана возможность подключения инструментов для диаграмм и структурного описания приложения.

Разработанное приложение соответствует варианту 8: пользователь вводит предложение, программа инвертирует слова и выводит результат. Дополнительно выполнены требования по отдельному окну ввода, восстановлению последнего значения, сообщению об ошибке и разделению классов по MVC с активной моделью.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода приложения

Main.java

```
package org.example;

import org.example.controller.WordInversionController;
import org.example.model.WordInversionModel;
import org.example.view.MainFrame;

import javax.swing.SwingUtilities;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            WordInversionModel model = new WordInversionModel();
            MainFrame view = new MainFrame();
            new WordInversionController(model, view);

            view.setVisible(true);
        });
    }
}
```

model/ModelListener.java

```
package org.example.model;

public interface ModelListener {
    void modelChanged(WordInversionModel model);
}
```

model/WordInversionModel.java

```
package org.example.model;

import java.util.ArrayList;
import java.util.List;

public class WordInversionModel {
    private final List<ModelListener> listeners = new ArrayList<>();
    private String sourceSentence = "";
    private String invertedSentence = "";

    public String getSourceSentence() {
        return sourceSentence;
    }

    public String getInvertedSentence() {
        return invertedSentence;
    }

    public void setSourceSentence(String sourceSentence) {
        this.sourceSentence = sourceSentence;
        this.invertedSentence = invertWords(sourceSentence);
        notifyListeners();
    }

    public void addListener(ModelListener listener) {
        listeners.add(listener);
    }
}
```

```

    }

    private void notifyListeners() {
        for (ModelListener listener : listeners) {
            listener.modelChanged(this);
        }
    }

    private String invertWords(String sentence) {
        StringBuilder result = new StringBuilder();
        StringBuilder word = new StringBuilder();

        for (int i = 0; i < sentence.length(); i++) {
            char symbol = sentence.charAt(i);
            if (Character.isLetterOrDigit(symbol)) {
                word.append(symbol);
            } else {
                appendReversedWord(result, word);
                result.append(symbol);
            }
        }

        appendReversedWord(result, word);
        return result.toString();
    }

    private void appendReversedWord(StringBuilder result, StringBuilder word) {
        if (word.isEmpty()) {
            return;
        }

        result.append(word.reverse());
        word.setLength(0);
    }
}

```

view/InputDialog.java

```

package org.example.view;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JDialog;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.Font;

public class InputDialog extends JDialog {
    private final JTextArea sentenceArea = new JTextArea(6, 32);
    private String result;

    public InputDialog(JFrame owner, String initialSentence) {
        super(owner, "Ввод данных", true);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        sentenceArea.setText(initialSentence);
        sentenceArea.setLineWrap(true);
        sentenceArea.setWrapStyleWord(true);
        sentenceArea.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 14));
    }
}

```

```

JLabel promptLabel = new JLabel("Введите предложение:");
JPanel contentPanel = new JPanel(new BorderLayout(8, 8));
contentPanel.setBorder(BorderFactory.createEmptyBorder(12, 12, 0, 12));
contentPanel.add(promptLabel, BorderLayout.NORTH);
contentPanel.add(new JScrollPane(sentenceArea), BorderLayout.CENTER);

JButton okButton = new JButton("OK");
JButton cancelButton = new JButton("Отмена");

okButton.addActionListener(event -> {
    result = sentenceArea.getText();
    dispose();
});
cancelButton.addActionListener(event -> {
    result = null;
    dispose();
});

JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
buttonPanel.setBorder(BorderFactory.createEmptyBorder(0, 12, 12, 12));
buttonPanel.add(okButton);
buttonPanel.add(cancelButton);

add(contentPanel, BorderLayout.CENTER);
add(buttonPanel, BorderLayout.SOUTH);

getRootPane().setDefaultButton(okButton);
pack();
setLocationRelativeTo(owner);
}

public String getResult() {
    return result;
}
}

```

view/MainFrame.java

```

package org.example.view;

import org.example.model.ModelListener;
import org.example.model.WordInversionModel;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.GridLayout;
import java.awt.event.ActionListener;

public class MainFrame extends JFrame implements ModelListener {
    private final JTextArea sourceArea = createDisplayArea();
    private final JTextArea resultArea = createDisplayArea();
    private final JButton inputButton = new JButton("Ввести данные");

    public MainFrame() {

```

```

super("Лабораторная работа 1. Вариант 8");
setDefaultCloseOperation(EXIT_ON_CLOSE);
setMinimumSize(new Dimension(620, 420));
setLayout(new BorderLayout(12, 12));

JLabel titleLabel = new JLabel("Утилита для инвертирования слов");
titleLabel.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 20));
titleLabel.setBorder(BorderFactory.createEmptyBorder(16, 16, 0, 16));

JPanel dataPanel = new JPanel(new GridLayout(2, 1, 10, 10));
dataPanel.setBorder(BorderFactory.createEmptyBorder(0, 16, 0, 16));
dataPanel.add(createTextBlock("Последний ввод:", sourceArea));
dataPanel.add(createTextBlock("Результат:", resultArea));

JPanel actionPanel = new JPanel(new BorderLayout());
actionPanel.setBorder(BorderFactory.createEmptyBorder(0, 16, 16, 16));
actionPanel.add(inputButton, BorderLayout.EAST);

add(titleLabel, BorderLayout.NORTH);
add(dataPanel, BorderLayout.CENTER);
add(actionPanel, BorderLayout.SOUTH);

pack();
setLocationRelativeTo(null);
}

public void setInputAction(ActionListener listener) {
    inputButton.addActionListener(listener);
}

public String showInputDialog(String initialSentence) {
    InputDialog dialog = new InputDialog(this, initialSentence);
    dialog.setVisible(true);
    return dialog.getResult();
}

public void showError(String message) {
    JOptionPane.showMessageDialog(this, message, "Ошибка ввода",
JOptionPane.ERROR_MESSAGE);
}

public void updateFromModel(WordInversionModel model) {
    sourceArea.setText(model.getSourceSentence());
    resultArea.setText(model.getInvertedSentence());
}

@Override
public void modelChanged(WordInversionModel model) {
    updateFromModel(model);
}

private JTextArea createDisplayArea() {
    JTextArea textArea = new JTextArea();
    textArea.setEditable(false);
    textArea.setLineWrap(true);
    textArea.setWrapStyleWord(true);
    textArea.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 14));
    return textArea;
}

private JPanel createTextBlock(String title, JTextArea textArea) {
    JPanel panel = new JPanel(new BorderLayout(6, 6));
    panel.add(new JLabel(title), BorderLayout.NORTH);
    panel.add(new JScrollPane(textArea), BorderLayout.CENTER);
}

```

```
        return panel;
    }
}
```

controller/WordInversionController.java

```
package org.example.controller;
```

```
import org.example.model.WordInversionModel;
import org.example.view.MainFrame;
```

```
public class WordInversionController {
    private final WordInversionModel model;
    private final MainFrame view;

    public WordInversionController(WordInversionModel model, MainFrame view) {
        this.model = model;
        this.view = view;

        this.model.addListener(view);
        this.view.setInputAction(event -> requestSentence());
        this.view.updateFromModel(model);
    }

    private void requestSentence() {
        String sentence = view.showInputDialog(model.getSourceSentence());

        if (sentence == null) {
            return;
        }

        if (sentence.trim().isEmpty()) {
            view.showError("Введите непустое предложение.");
            return;
        }

        model.setSourceSentence(sentence);
    }
}
```